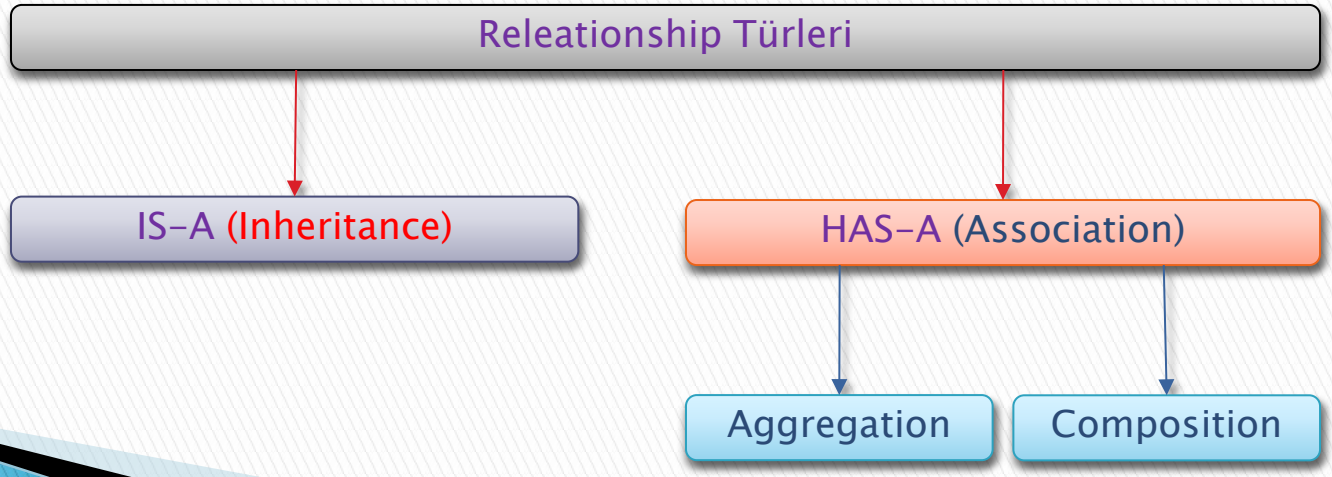
Öğr.Gör. Mustafa GÖKMEN
gokmen@selcuk.edu.tr

Nesneye Yönelik Programlama

OOP
Associations

Relationship (İlişki)

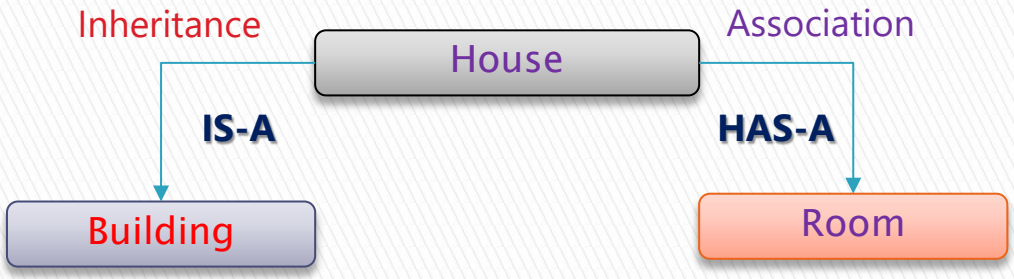
- Bir sınıf içerisinde, başka bir sınıfa ait olan nesne referansı ile o sınıfa ait değişken ve metodlar doğrudan ilişkilendirilerek kullanılabilmektedir.
- Sınıflar arasındaki ilişkiyi tanımlamanın en basit yolu iki veya daha fazla sınıf arasındaki işbirliğinin şeklini mantıksal olarak belirlemektir. Bu mantıksal ilişki, nesnelerin birbirleriyle nasıl iletişim kurduklarını ve ilişkideki nesneler tarafından sağlanan işlevleri ve hizmetleri nasıl kullandıkları gösterir.
- İki tür ilişkiden bahsedilebilir:
  - IS-A ilişkisi (Inheritance)
  - HAS-A ilişkisi (Association)



İlişki Türleri

- IS-A ilişkisi (Inheritance), iki nesne arasındaki ilişkinin kalıtım yoluyla sağlandığını gösterir.
  - ✓ Örneğin; "Kartal bir kuştur." ("Eagle is a bird.") dediğimizde Kartal'ın, Kuş sınıfından (inherited-kalıtılmış) türetilmiş olduğunu anlarız.

- HAS-A ilişkisi (Association), iki bağımsız nesne arasındaki mantıksal ilişkiyi ifade eder.
  - ✓ Örneğin; "Evin odası var." (House has a room.") dediğimizde ev ile oda arasındaki mantıksal ilişkiyi tarif etmiş oluruz.

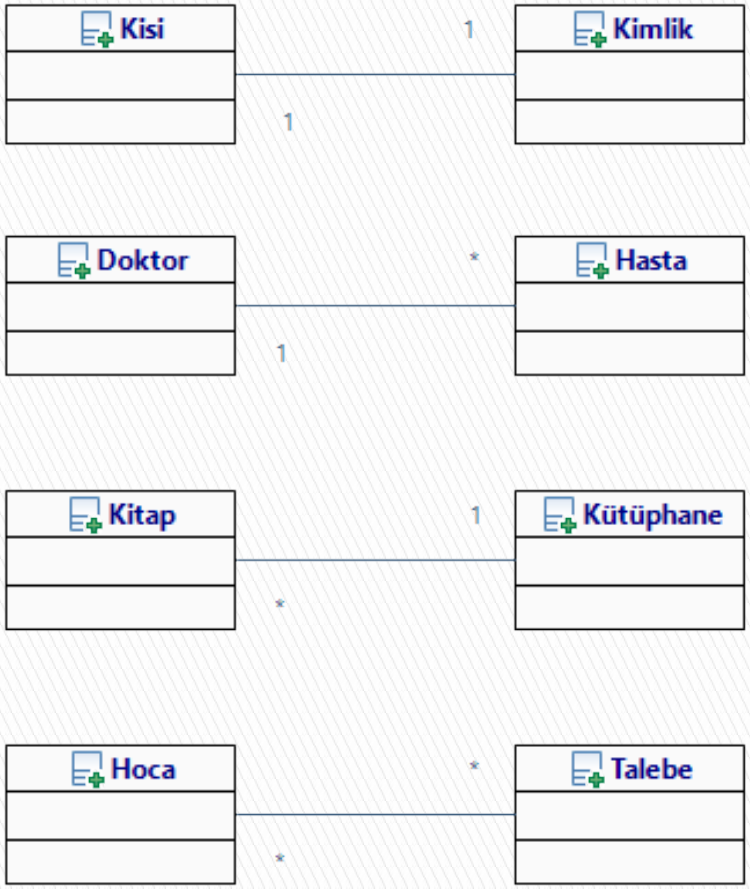


Association

- Etkileşimdeki nesnelerin çokluğunu, ilişkili öğelerin yanlarında "Multiplicity (Çokluk)" işaretleri ekleyerek de gösterilebilir.

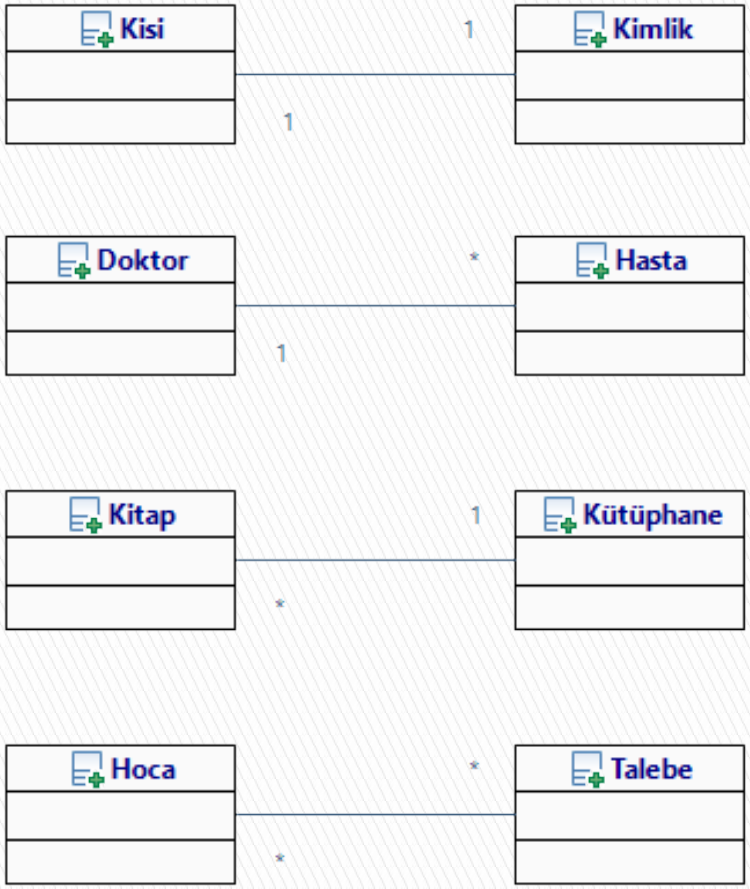
- Multiplicity, sınıftan kaç tane nesnenin ilişkiye dahil olduğunu belirten bir sayıyı belirtir.

- UML diyagramında, sadece mantıksal bağlantı gösterilecek ise, ilişkinin sahibi (Owner); "Association" olarak seçilir. Ancak fiziksel olarak (kod üretimi vb. için) bağlantı da gösterilecekse ilişkinin sahibi (Owner); "Classifier" ve ayrıca "Navigable" olarak ayarlanmalıdır.



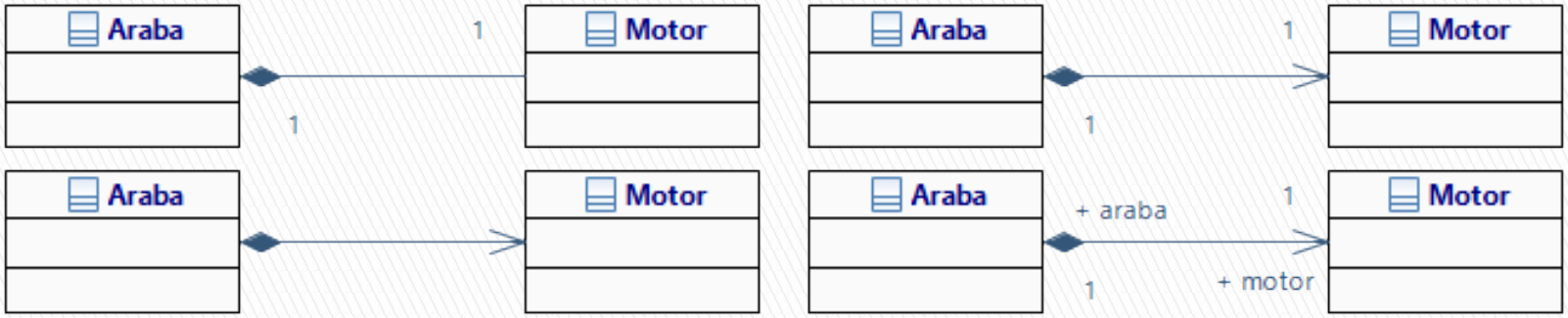
Association

Multiplicity	Anlamı
1..1	Sadece bir (Exactly one) UML'de tek rakamla 1 şeklinde gösterilir.
0..*	Hiç yok veya en az bir (none or more)
0..1	Hiç ya da en fazla bir (none or one)
1..*	En az bir veya daha çok (at least one or more)
m..m	Sadece m tane (Exactly m) Meseala 2..2 ise sadece 2 tane. Bu örnek; UML'de tek rakamla 2 şeklinde gösterilir.
m..n	En az m, en fazla n tane (at least m but no more than n) Mesela: 2..4: en az iki ve en fazla 4 tane



Association Türleri

- Association, mantıksal yapılandırmaya göre iki farklı alt türe daha sahiptir:
  - Aggregation (part-of relationship)
  - Composition (whole-part relationship)
- Association, sınıflar arasındaki çokluk (Multiplicity) durumunu gösteren ifadeler ile birlikte oksuz olarak gösterilebilir.
- Aynı zamanda, ilişkinin yönünü gösteren ucu açık bir ok ile biten düz çizgi şeklinde de temsil edilebilir.
  - Okun yönü bütün(whole)den parça(part)ya doğrudur.



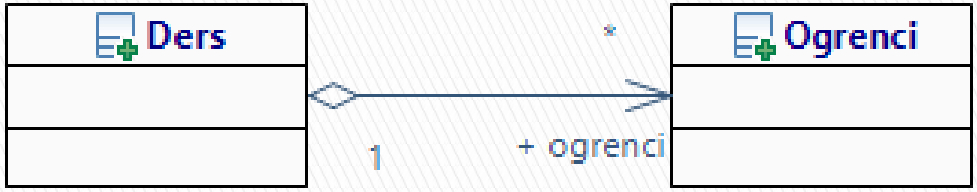
Aggregation (Bağlılık)

Birbirlerinden bağımsız nesnelerin "part-of relationship (İlişkinin parçası olma)" şeklindeki ilişkisi aggregation olarak ifade edilir.

Aggregation (Empty diamond)

Örnek1:

- "Derste öğrenciler var." cümlesinde; Ders süresince öğrenciler dersin bir parça(part-of)sıdır. Ders, hiçbir şekilde öğrencilerin sahibi değildir. Ders ve öğrenci varlıkları birbirinden bağımsız birer nesnedir. Buna göre; ders sona erdiğinde öğrenciler yine de var olmaya devam edecektir.



Örnek2:

- "Arabanın radyosu var" cümlesinde; radyo araba'nın bir parça(part-of)sıdır. Araba radyosuz ve radyo da arabasız var olmaya devam edebilir. Arabanın radyosu olmak zorunda değildir ve bu durum arabanın da radyonun da durumunu değiştirmez.

Composition (Bağımlılık)

"whole-part (bütünlük)" ilişkisini temsil eden bir ilişki biçimidir. Bir bütünü oluşturan parçaların varlığının, bütünün varlığına tek yönlü olarak bağımlı olması composition olarak tarif edilir. Daha kısıtlayıcı aggregation türüdür.

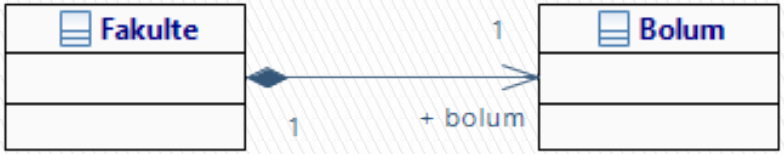
Composition (Filled diamond)

Örnek1:

- "Evin odaları vardır" cümlesinde; Ev(whole), odalar(part)'ın sahibidir. Yani; evin odaları vardır. Odalar, ev'den bağımsız değildir ve ev yıkılırsa, doğal olarak odalar da yıkılmış olacaktır.

Örnek2:

- "Fakültenin bölümü vardır" cümlesinde Fakülte(whole) bölüm(part)'ün sahibidir. Bölüm Fakülteden bağımsız değildir ve Fakülte kapatılırsa doğal olarak bölüm de kapatılmış oacaktır.



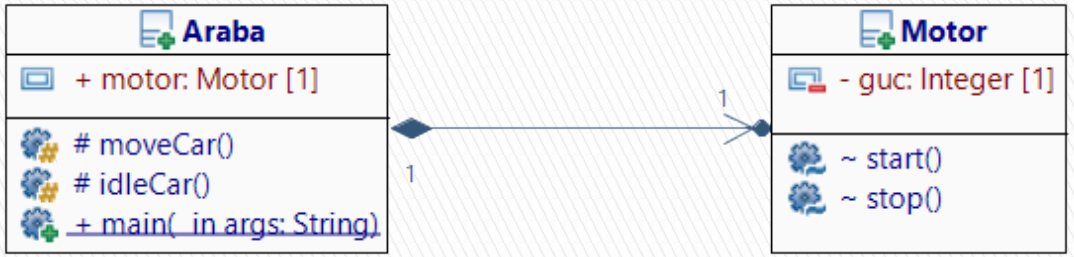
Örnek: Araba (v1.0)

```java
public class Araba {
  public Motor motor = new Motor();

  protected void moveCar() {
    motor.start();
    System.out.println("Araba hareket ediyor.");
  }

  protected void idleCar() {
    System.out.println("Araba durdu.");
    motor.stop();
  }

  public static void main(String[] args) {
    Araba araba = new Araba();
    araba.moveCar();
    araba.idleCar();
  }
}
```

Araba'da Motor tipinde bir **Nesne Değişkeni** tanımlanmış durumda.

```java
public class Motor {
  private int guc = 100;

  void start() {
    System.out.println(guc + " HP motor calisti.");
    }

  void stop() {
    System.out.println(guc + " HP motor durdu.");
  }
}
```

```
100 HP motor calisti.
Araba hareket ediyor.
Araba durdu.
100 HP motor durdu.
```



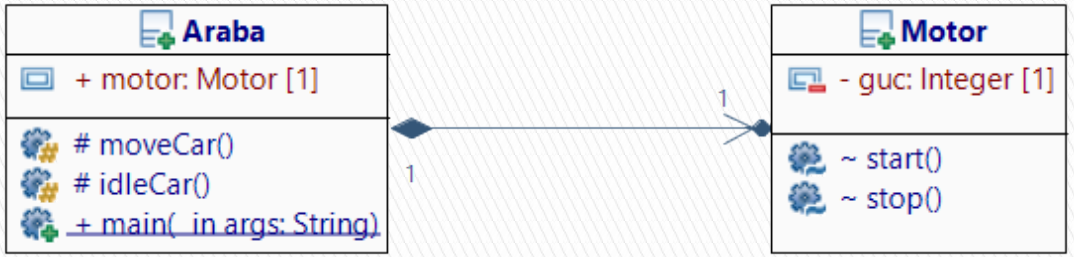
Örnek: Araba (v2.0)

```java
public class Araba {
  public Motor motor = new Motor();

  protected void moveCar() {
    System.out.println("Araba hareket ediyor.");
  }

  protected void idleCar() {
    System.out.println("Araba durdu.");
  }

  public static void main(String[] args) {
    Araba araba = new Araba();
    araba.motor.start();
    araba.moveCar();
    araba.idleCar();
    araba.motor.stop();
  }
}
```

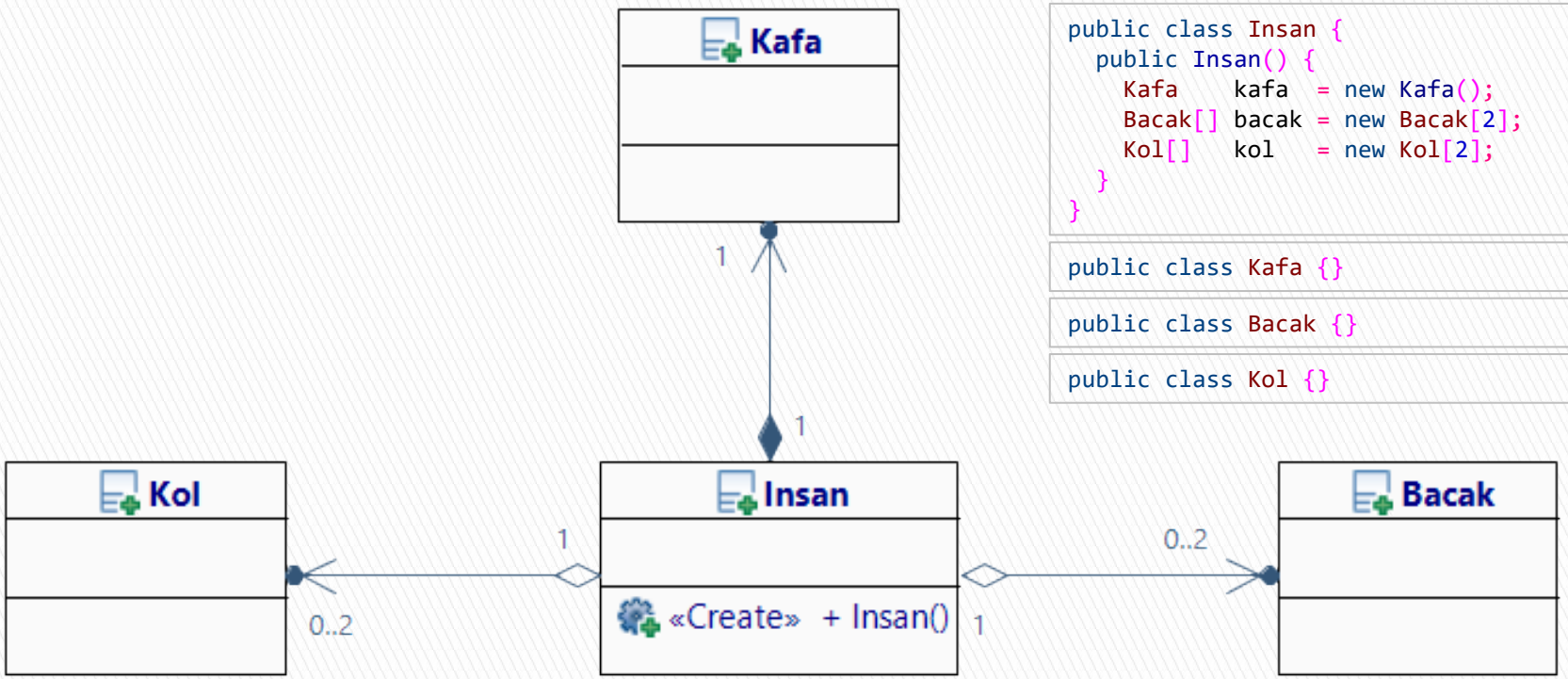
Araba'da Motor tipinde bir **Nesne Değişkeni** tanımlanmış durumda.

```java
public class Motor {
  private int guc = 100;

  void start() {
    System.out.println(guc + " HP motor calisti.");
    }

  void stop() {
    System.out.println(guc + " HP motor durdu.");
  }
}
```

```
100 HP motor calisti.
Araba hareket ediyor.
Araba durdu.
100 HP motor durdu.
```



Aggregation-Composition



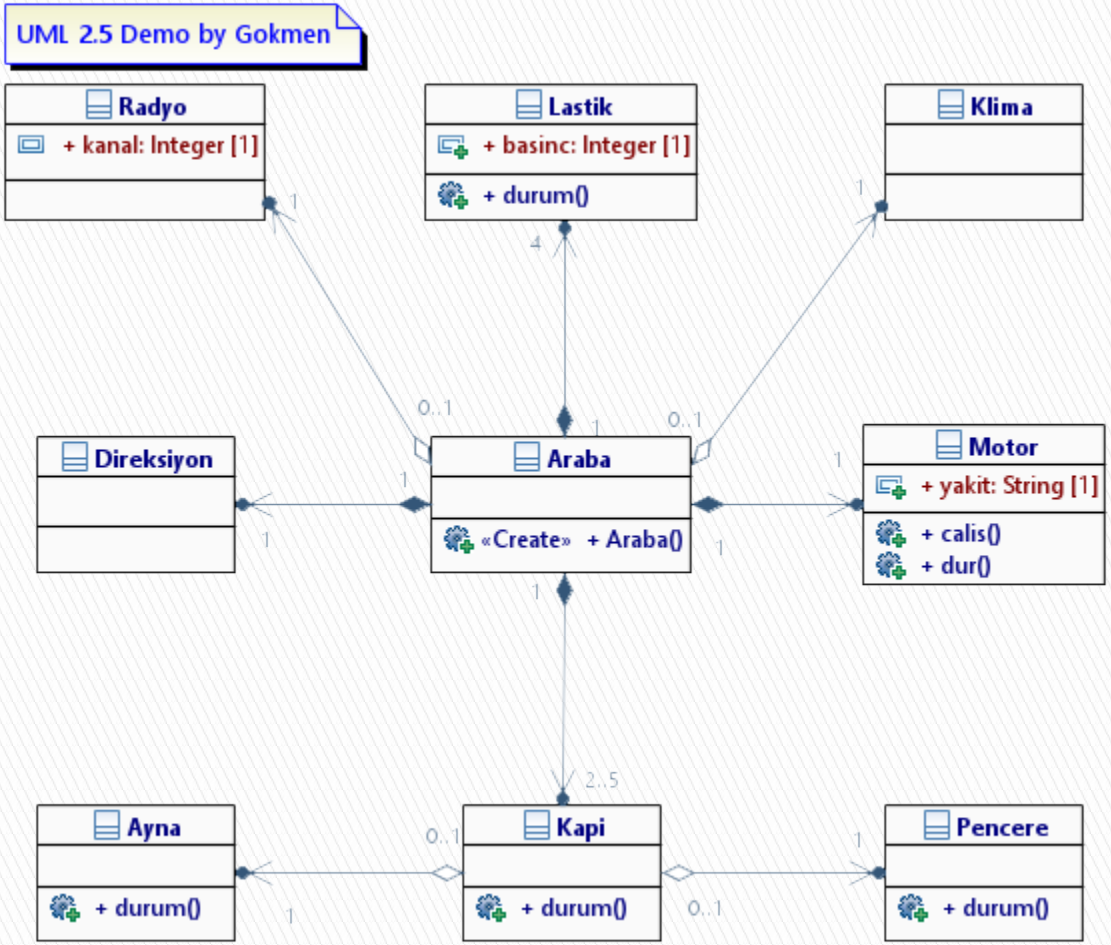

```
public class Insan {
  public Insan() {
    Kafa    kafa  = new Kafa();
    Bacak[] bacak = new Bacak[2];
    Kol[]   kol   = new Kol[2];
  }
}
```

```
public class Kafa {}
```

```
public class Bacak {}
```

```
public class Kol {}
```

Aggregation : ❓

Composition : ❓

Association Uygulaması